

Benchmarking Local Large Language Models for Structured Knowledge Extraction in Conversational Commerce

Ratomir Karlović¹ , Luka Sever¹ , Sandi Baressi Šegota¹ , Vedran Mrzljak²  and Ivan Lorencin¹ 

¹ Faculty of Informatics, Juraj Dobrila University of Pula, Zagrebačka 30, 52100 Pula, Croatia; ratomir.karlovic@unipu.hr (R.K.), luka.sever@unipu.hr (L.S.), sandi.baressi.segota@unipu.hr (S.B.Š.), ivan.lorencin@unipu.hr (I.L.)

² Faculty of Engineering Rijeka, University of Rijeka, Vukovarska 58, 51000 Rijeka, Croatia;

* Correspondence: vedran.mrzljak@riteh.uniri.hr;

Abstract

The integration of natural language understanding into conversational commerce requires robustly extracting structured intents from idiomatic text. While cloud-based models are highly capable, their latency, privacy risks, and operational costs necessitate efficient local alternatives. To address this, we empirically evaluate locally deployed, open-weights large language models (LLMs) on parsing multi-intent shopping cart commands into deterministic JSON schemas. We benchmark multiple models across minimal, extended, and few-shot prompting paradigms, utilizing a dual-axis framework to simultaneously measure extraction quality (operation F1-score, exact match, schema validity) and hardware performance (latency, GPU power consumption, VRAM usage). Our evaluation highlights a fundamental trade-off between schema compliance and computational overhead, demonstrating how advanced prompting techniques improve parsing accuracy while measurably impacting inference latency and power draw. Ultimately, this comprehensive profiling identifies optimal model configurations for resource-constrained environments, establishing a practical baseline for deploying cost-effective and energy-efficient LLMs in edge-based retail automation.

Keywords: LLM; Prompt Engineering; Natural Language Understanding; JSON Parsing; Structured Output; Model Performance; Conversational Agents; Information Extraction;

1. Introduction

The template details the sections that can be used in a manuscript. Note that the order and names of article sections may differ from the requirements of the journal (e.g., the positioning of the Materials and Methods section). Please check the instructions on the authors' page of the journal to verify the correct order and names. For any questions, please contact the editorial office of the journal or support@mdpi.com. For LaTeX-related questions please contact latex@mdpi.com.

2. Background and Related Work

3. Methodology

We evaluate local large language models on a single structured-extraction task using a *dual-axis* protocol that scores every model on both extraction quality and computational cost. The task, dataset, and serving stack are held fixed; only the model and the prompting regime

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the

[Creative Commons Attribution \(CC BY\)](#) license.

vary. No model is fine-tuned—all are used off-the-shelf through in-context prompting—so the study isolates the behaviour of pretrained open-weights models under realistic, low-effort local deployment. The remainder of this section defines the extraction task (Section 3.1), the evaluation corpus (Section 3.2), the model pool (Section 3.3), the three prompting regimes (Section 3.4), the inference and measurement protocol (Section 3.5), and the quality and cost metrics (Section 3.6). The prompts are reproduced verbatim in Appendix A.

3.1. Task Formulation

Given a single natural-language utterance addressed to a shopping assistant, a model must return the set of cart operations the utterance encodes. Each operation is a triple (*action*, *product*, *quantity*) in which *action* ∈ {add, remove}, *product* is the product name as it appears in the utterance (with action verbs and numerals removed), and *quantity* ∈ ℤ_{≥1}, defaulting to 1 when the user gives no number. The required output is a JSON array of such triples, for example [{"action": "add", "product": "pies", "quantity": 20}]. The task is intentionally demanding along three dimensions: (i) one utterance bundles several operations that must be jointly segmented and recovered; (ii) intent is conveyed through open-ended synonyms (“toss in”, “ring up”, “nix”, “take out”) that must be collapsed onto the two canonical actions; and (iii) quantities are written as digits, number words, and idioms—“a pair” → 2, “half a dozen” → 6, “two dozen” → 24—that must be normalised to integers. A correct response therefore couples intent classification, verbatim span extraction, and quantity normalisation, emitted as schema-conformant JSON in a single forward pass.

3.2. Evaluation Corpus

We evaluate on a purpose-built corpus of 5000 shopping-cart commands, each paired with a gold list of operations. The corpus was generated with a frontier large language model (Anthropic Claude Opus 4.8 at maximum reasoning effort), prompted to emit colloquial, lexically varied utterances together with their exact structured parse; a randomly drawn 5% subset (250 commands) was then manually inspected to confirm that the gold operations matched the utterance. The corpus is used solely for evaluation: because no model is trained or tuned, there is no train/test split, and every command contributes to the reported metrics.

Table 1 summarises the corpus. It spans 149 distinct retail products and 18,175 gold operations, a mean of 3.64 operations per command, with every command carrying between two and five. Additions outnumber removals (62.6% vs. 37.4%), and quantities are integers in [1, 25] with a mean of 11.0. Because each utterance carries several operations and mixes additions with removals, exact recovery demands that the model track multiple intents at once rather than predict a single dominant action.

Table 1. Composition of the evaluation corpus: distribution of commands by the number of gold operations they contain (5000 commands, 18,175 operations, mean 3.64 per command).

Operations per command	2	3	4	5	Total
Commands	427	1840	1864	869	5000
Share (%)	8.5	36.8	37.3	17.4	100.0

3.3. Models

We benchmark six open-weights, instruction-tuned models that span roughly an order of magnitude in scale (from 3B to 31B parameters) and five families—Meta Llama, Google Gemma, Mistral, IBM Granite, and OpenAI gpt-oss (Table 2). Every model was obtained from the Ollama library using its default tag and the default quantization published for that tag (predominantly 4-bit; gpt-oss in its native MXFP4 format); none was re-quantized

or fine-tuned. This reproduces the configuration a practitioner obtains from a single `ollama pull`, which is precisely the low-effort, on-premises deployment scenario the paper targets.

Table 2. Evaluated models, all served through Ollama (v0.21.2-rc0) using the default tag and quantization. Parameter counts are the nominal sizes indicated by the model tag; “MoE” marks a mixture-of-experts model.

Ollama tag	Developer / family	Params	Architecture
<code>gemma4:31b</code>	Google / Gemma	31B	Dense
<code>gpt-oss:20b</code>	OpenAI / gpt-oss	20B	MoE
<code>llama3.1:8b</code>	Meta / Llama 3.1	8B	Dense
<code>mistral:7b</code>	Mistral AI / Mistral	7B	Dense
<code>granite4:3b</code>	IBM / Granite 4.0	3B	Dense
<code>llama3.2:3b</code>	Meta / Llama 3.2	3B	Dense

3.4. Prompting Regimes

Each model is run under three system prompts of increasing specification while the user turn (the raw command) is held identical, so that any difference in output is attributable to the prompt alone. The *minimal* prompt states the task and the three output fields in a few lines. The *extended* prompt adds explicit instructions—chiefly that synonymous verbs must be mapped onto `add/remove` and that an unspecified quantity defaults to one—but gives no examples. The *few-shot* prompt instead pairs the schema with three worked input–output exemplars that demonstrate synonym mapping, idiomatic-quantity normalisation (“a pair” → 2, “a dozen” → 12), and the single-item default. Contrasting minimal with extended isolates the marginal value of *instruction detail*, whereas contrasting extended with few-shot isolates the marginal value of *in-context demonstration*. The full prompt texts appear in Appendix A.

3.5. Inference and Measurement Protocol

All models were served locally with Ollama (v0.21.2-rc0) on a single compute node providing two NVIDIA A100-SXM4 GPUs (40GB each, 80GB aggregate), a 32-core AMD EPYC 7763 CPU, and 120GB of system memory; the runtime shards the larger models across both GPUs. Each command was issued once, using Ollama’s default decoding parameters—no temperature, top-*p*, context-length, or random-seed overrides—so that the measurements reflect default out-of-the-box behaviour. Generation is consequently stochastic, and each of the $6 \times 3 = 18$ model–prompt configurations is evaluated in a single pass over the 5000 commands (90,000 generations in total); we do not average over repeated runs and therefore treat run-to-run variance as a limitation of the design.

For every request we record the wall-clock latency around the generation call, the prompt- and output-token counts reported by the runtime, and the generation time. GPU telemetry is sampled from `nvidia-smi` immediately after each request: power draw and utilisation are averaged across the two GPUs, while memory usage is summed. From the aggregated samples we derive a first-order per-command energy estimate, $E \approx \bar{P} \bar{t}$, the product of mean (per-GPU) power $\bar{P}$ and mean latency $\bar{t}$; we report it as an indicative figure rather than a metered measurement.

3.6. Evaluation Metrics

Each configuration is scored on a quality axis and a cost axis, both computed directly from the stored responses.

Response parsing. Output is first reduced to operations by a lenient parser: we attempt a direct JSON parse and, on failure, extract the longest bracket-delimited substring (first `[` to last `]`, or first `{` to last `}`) and re-parse. This tolerates incidental wrapping such as prose preambles or Markdown code fences, so JSON validity credits any response from

which a structured payload can be recovered; the schema and content metrics below then impose the stricter requirements.

Quality metrics. (i) *JSON validity*—the fraction of responses yielding a parseable payload. (ii) *Schema validity*—the fraction whose parsed operations are all well-typed objects carrying exactly the keys *action* ($\in \{\text{add}, \text{remove}\}$), *product* (string), and *quantity* (integer); an empty result is invalid. (iii) *Operation-level micro-averaged F1* (Op-F1), the primary quality scalar: each operation is treated as the exact triple (*action*, *product*, *quantity*), and for each command we take the multiset overlap of predicted and gold operations, accumulating true positives, false positives, and false negatives over the entire corpus before computing a single micro-averaged precision, recall, and F1. Op-F1 is order-independent but requires all three fields to match exactly. (iv) *Exact match* (EM)—the fraction of commands whose predicted operation list equals the gold list exactly and in order, a strict whole-utterance criterion. (v) *Field accuracies* for action, product, and quantity—predicted and gold operations are aligned by position up to the longer length, and we report the fraction of aligned slots whose field agrees; being position-aligned, these conflate ordering and count errors with genuine field errors and are reported only as a diagnostic decomposition.

Cost metrics. Per configuration we report latency (mean, median, and 95th percentile, in seconds), throughput (commands per second over the run), mean GPU power (W , averaged over the two GPUs), per-command energy ($E \approx \bar{P} \bar{t}$, J), mean resident VRAM (GB, summed over both GPUs), and the mean prompt- and output-token counts. For the trade-off analysis (Section 4.3) quality is summarised by Op-F1 and cost by energy per command, the two scalars that anchor the Pareto comparison.

Generative-AI disclosure. In accordance with MDPI policy, we disclose that generative AI was used in a single part of this study—to synthesise the evaluation corpus (Section 3.2)—and nowhere else; model evaluation, measurement, analysis, and manuscript preparation used no generative-AI assistance beyond superficial text editing.

4. Results and Discussion

We evaluate six open-weights models under three prompting regimes—*minimal*, *extended*, and *few-shot*—yielding 18 model-prompt configurations, each applied to the full set of 5000 shopping-cart commands. These commands contain a mean of 3.64 atomic operations (18,175 gold operations in total), so a single utterance typically bundles several add/remove intents that must be jointly recovered. Following the dual-axis protocol introduced in the Methodology, we report extraction quality (Table 3) and computational cost (Table 4) for every configuration, and then analyse the quality–cost trade-off that the prompting regime induces (Figure 6). Throughout, we use the operation-level micro-averaged F1 score (Op-F1) as the primary quality scalar and an estimated energy per command, $E \approx \bar{P} \bar{t}$ (mean GPU power $\times$ mean latency), as the primary cost scalar.

4.1. Extraction Quality and the Role of the Prompting Regime

148

Table 3. Extraction quality by model and prompting regime (Min, minimal; Ext, extended; FS, few-shot). JSON and Schema denote the JSON- and schema-validity rates; Op-F1 is the operation-level micro-averaged F1 score; EM is the ordered full-command exact-match rate. All values are computed over the 5000-command evaluation set. The best Op-F1 per model is shown in bold.

Model	Prompt	JSON	Schema	Op-F1	EM
Gemma4 31B	Min	1.000	1.000	0.951	0.833
	Ext	1.000	1.000	0.967	0.885
	FS	1.000	1.000	0.981	0.933
GPT-OSS 20B	Min	1.000	1.000	0.923	0.753
	Ext	0.999	0.999	0.944	0.819
	FS	1.000	1.000	0.975	0.915
Llama3.1 8B	Min	0.935	0.881	0.782	0.538
	Ext	0.998	0.976	0.801	0.568
	FS	0.997	0.995	0.950	0.841
Mistral 7B	Min	0.963	0.877	0.689	0.357
	Ext	0.982	0.865	0.504	0.281
	FS	0.993	0.976	0.942	0.804
Granite4 3B	Min	1.000	0.928	0.797	0.512
	Ext	1.000	0.931	0.790	0.516
	FS	1.000	0.997	0.937	0.807
Llama3.2 3B	Min	0.967	0.606	0.610	0.311
	Ext	0.970	0.637	0.526	0.229
	FS	0.890	0.822	0.813	0.566

Few-shot prompting yields the best Op-F1 for all six models, and the magnitude of the gain is inversely related to the model’s zero-shot capability. Relative to the minimal prompt, in-context demonstrations raise Op-F1 by only +0.031 for Gemma4 31B and +0.051 for GPT-OSS 20B, but by +0.140 (Granite4 3B), +0.168 (Llama3.1 8B), +0.203 (Llama3.2 3B), and +0.253 (Mistral 7B) for the smaller models. The exemplars thus act as a capability equaliser: models that are mediocre zero-shot parsers become competitive once shown the target format, whereas the largest models—already near ceiling—have little headroom to gain. Figure 1 visualises this compression of the quality gap between small and large models.

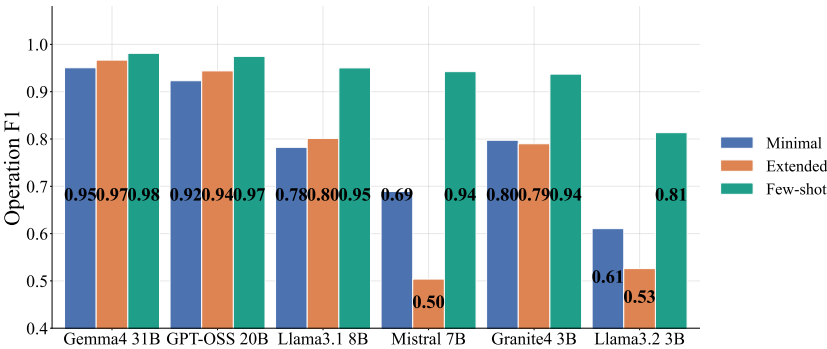


Figure 1. Operation-level F1 by model and prompting regime. Few-shot prompting (teal) yields the best accuracy for all six models and sharply compresses the gap between small and large models; the extended prompt (orange) is inconsistent, degrading Mistral 7B and the Llama3.2 3B small model relative to the minimal prompt (blue).

Decomposing quality into format adherence and content correctness clarifies the mechanism. Under the minimal prompt, several small models emit syntactically valid JSON that nonetheless violates the target schema: schema validity is only 0.928 (Granite4 3B), 0.881 (Llama3.1 8B), and 0.606 (Llama3.2 3B) despite JSON validity at or above 0.94. Few-

shot demonstrations repair this format gap almost entirely—schema validity rises to 0.997, 0.995, and 0.822, respectively—indicating that the dominant failure mode of small models is structural (key naming, value typing, nesting) rather than semantic. Demonstrations specify the output contract by example rather than by description, which the smaller models follow more reliably than prose instructions. Figure 2 makes the format-versus-content split explicit: JSON validity sits at or near the ceiling for almost every configuration, so the small-model failures under weaker prompts are schema violations rather than malformed JSON (the corresponding schema-validity rates appear in Table 3).

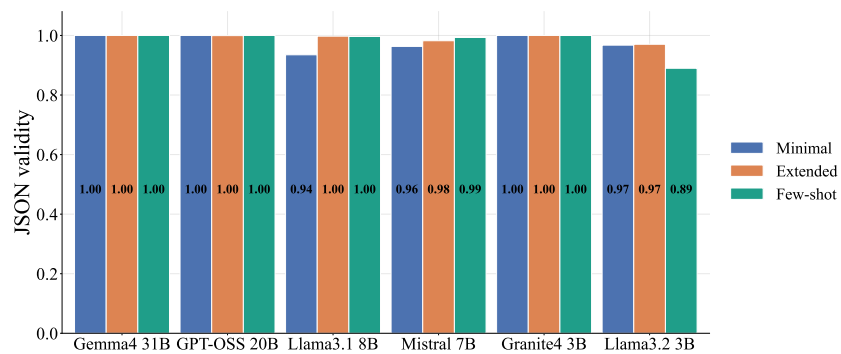


Figure 2. JSON validity by model and prompting regime (bars: minimal, extended, few-shot). Across almost all configurations the models emit parseable JSON at or near the ceiling; the residual small-model deficits under weaker prompts are predominantly schema violations rather than malformed JSON (cf. the schema-validity column of Table 3).

In contrast, the extended (instruction-rich) prompt is not a monotonic improvement and is frequently harmful. It helps only the most capable models—Gemma4 31B (+0.016), GPT-OSS 20B (+0.021), and Llama3.1 8B (+0.019)—while degrading Mistral 7B (−0.185) and Llama3.2 3B (−0.084), and leaving Granite4 3B essentially unchanged (−0.007). Manual inspection traces the Mistral regression to over-normalisation: the additional instructions induce the model to lemmatise product spans (“toothpastes” → “toothpaste”, “quinoa bags” → “quinoa bag”) and to misresolve idiomatic quantities (“a pair” → 1), collapsing product-field accuracy from 0.762 to 0.559 even though the emitted JSON remains well-formed. Longer instructions therefore introduce degrees of freedom that smaller models resolve in unintended, ostensibly “helpful” ways that diverge from the verbatim-span ground truth. This is an important negative result: scaling prompt complexity without grounding it in exemplars can reduce extraction fidelity.

4.2. Computational Cost and the Decode-Bound Regime

182

Table 4. Computational cost by model and prompting regime. Lat_{med} , Lat_{mean} , and Lat_{p95} are latency percentiles (s); Thr is throughput (commands/s); Pwr is mean GPU power (W); E is the estimated energy per command (J , $E \approx \bar{P} \bar{t}$); P-tok and E-tok are the mean prompt- and output-token counts. Lower is better for all columns except throughput.

Model	Prompt	Lat_{med}	Lat_{mean}	Lat_{p95}	Thr	Pwr	E (J)	P-tok	E-tok
Gemma4 31B	Min	13.03	16.33	37.24	0.06	156	2555	146	600
	Ext	13.25	16.27	36.62	0.06	156	2545	197	599
	FS	10.10	11.71	19.87	0.08	156	1820	436	426
GPT-OSS 20B	Min	2.46	2.54	3.90	0.36	124	315	196	345
	Ext	2.44	2.52	3.95	0.36	124	313	248	341
	FS	2.39	2.51	3.89	0.37	126	315	455	339
Llama3.1 8B	Min	0.72	0.71	0.98	1.06	141	101	138	79
	Ext	0.64	0.66	0.93	1.12	141	94	191	72
	FS	0.58	0.56	0.71	1.26	140	79	402	57
Mistral 7B	Min	0.61	0.59	0.84	1.24	146	87	148	88
	Ext	0.58	0.56	0.78	1.30	147	82	200	82
	FS	0.50	0.49	0.64	1.42	143	70	467	70
Granite4 3B	Min	0.65	0.61	0.82	1.21	119	73	136	98
	Ext	0.53	0.52	0.81	1.35	118	62	189	81
	FS	0.41	0.39	0.51	1.64	118	46	400	56
Llama3.2 3B	Min	0.51	0.53	0.76	1.34	123	65	148	90
	Ext	0.50	0.49	0.63	1.42	124	61	201	80
	FS	0.40	0.39	0.48	1.67	122	47	412	56

The cost profile is governed by decoding rather than prefill. Although few-shot prompts are three- to fourfold longer on the input side (prompt tokens rise from ~ 140 to ~ 400 – 470), they consistently reduce wall-clock latency because the demonstrations suppress verbose generation: mean output length falls (e.g., $98 \rightarrow 56$ eval tokens for Granite4 3B and $600 \rightarrow 426$ for Gemma4 31B), and decode dominates latency in autoregressive inference. Consequently, few-shot mean latency is 17.7–35.6% lower than minimal for five of the six models (Granite4 3B –35.6%, Gemma4 31B –28.3%, Llama3.2 3B –27.2%, Llama3.1 8B –21.2%, Mistral 7B –17.7%), with GPT-OSS 20B effectively flat (–1.4%). The conventional intuition that richer prompts cost more compute is thus inverted here: by constraining the output, exemplars make inference simultaneously more accurate and faster. Figures 3 and 4 contrast the prompt and output token counts that drive this effect.

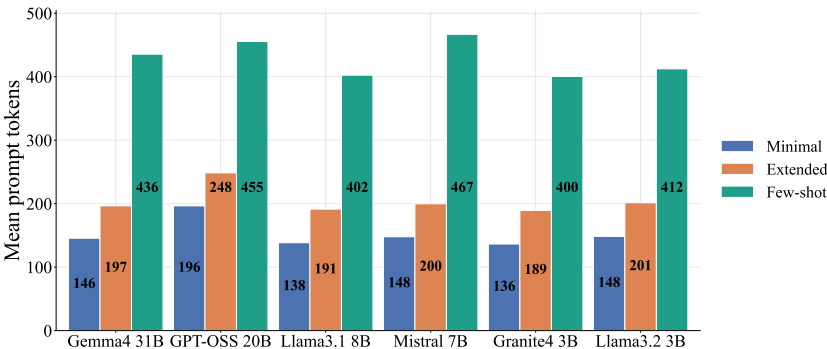


Figure 3. Mean prompt (input) tokens per command, by model and prompting regime. Few-shot prompts are markedly longer on the input side because each carries three worked exemplars, whereas the minimal and extended prompts are far shorter.

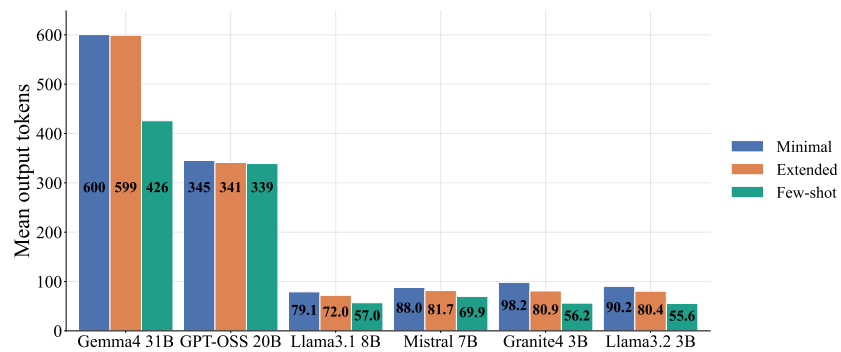


Figure 4. Mean output (decoded) tokens per command, by model and prompting regime. Despite the longer input, few-shot prompting yields *shorter* output: the demonstrations elicit concise, minified JSON and suppress verbose generation. Because decoding dominates latency, this reduction in output tokens is the mechanism behind the lower few-shot latency and energy reported in Table 4.

GPU power draw is essentially model-bound rather than prompt-bound—within each model it varies by only a few watts across regimes (e.g., ~156 W for Gemma4 31B, ~124 W for GPT-OSS 20B, ~118 W for Granite4 3B)—so the estimated energy per command tracks latency (Figure 5). Few-shot prompting therefore reduces per-command energy by 19–36% for the capable models, with the smallest models reaching 46 J (Granite4 3B) and 47 J (Llama3.2 3B) per command versus 315 J (GPT-OSS 20B) and 1820 J (Gemma4 31B). Energy spans a factor of roughly 40 across the frontier—a far wider dynamic range than quality—which is the crux of the deployment trade-off analysed in Section 4.3.

We report mean VRAM occupancy in the underlying logs for completeness but caution against ranking models by it: the measured footprint does not track parameter count (the 20B GPT-OSS model occupies only ~16 GB, less than the 3B Llama3.2 at ~17 GB or the 8B Llama3.1 at ~22 GB), which indicates that the figure reflects server-level allocator reservation and key–value-cache pre-allocation under the serving runtime rather than the minimal model footprint. Absolute VRAM values should accordingly be read as an upper bound on residency for this deployment, not as an intrinsic model property.

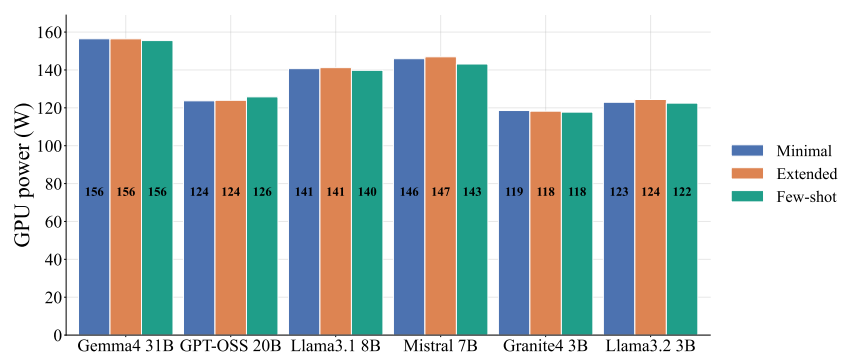


Figure 5. Mean GPU power draw (averaged across the two A100 GPUs) by model and prompting regime (bars: minimal, extended, few-shot). Power is essentially model-bound: within each model it is nearly invariant across prompting regimes, so per-command energy ($E \approx \bar{P} \bar{t}$) is governed by latency rather than by power. The remaining cost metrics—latency, throughput, energy, VRAM, and token counts—are reported in Table 4.

4.3. The Quality–Cost Trade-off

209

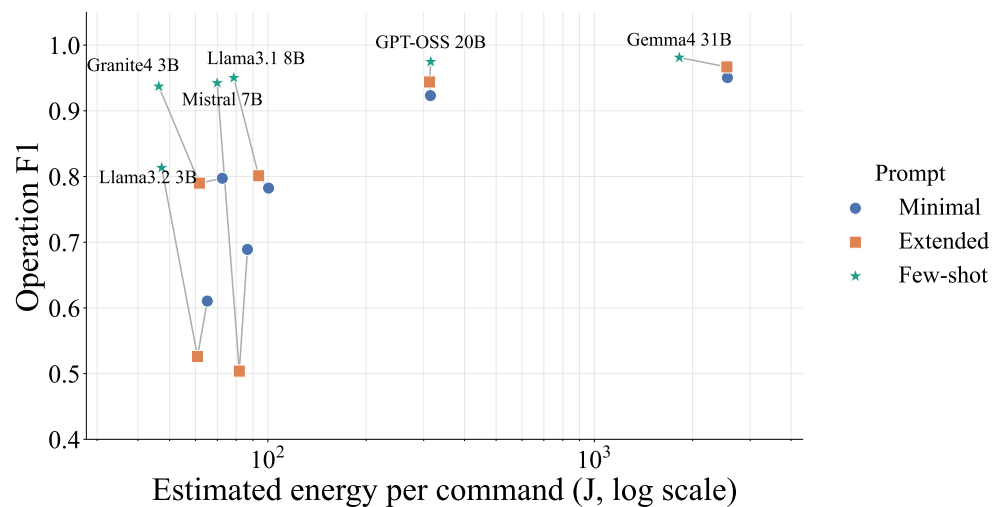


Figure 6. Quality–cost trade-off across all 18 configurations. Each point is one model–prompt configuration; thin grey lines connect the three regimes of a model. The horizontal axis is the estimated energy per command on a logarithmic scale. For every model the few-shot configuration (star) lies above and to the left of its minimal (circle) and extended (square) counterparts—simultaneously more accurate and more energy-efficient—so the operative trade-off is across model scale along the few-shot frontier rather than across prompts.

Figure 6 plots every configuration in the energy–quality plane and reveals the central finding of this study: for capable models, few-shot prompting is Pareto-improving rather than Pareto-trading—it moves each model up (higher F1) and to the left (lower energy) at once. The few-shot points dominate their own minimal and extended counterparts for all six models. The genuine trade-off is therefore not between prompts but across model scale: along the few-shot frontier, quality and cost are positively coupled, and the practitioner’s decision reduces to selecting an operating point on that frontier. Three operating points are practically distinct:

- **Maximum accuracy.** Gemma4 31B (few-shot) attains the highest fidelity (Op-F1 0.981, exact match 0.933) but at 1820 J and 11.7 s per command—appropriate only where accuracy is paramount and latency and energy are unconstrained, such as offline or server-side processing.
- **Balanced.** GPT-OSS 20B (few-shot) reaches Op-F1 0.975 at 315 J and 2.5 s, a 5.8× energy reduction for a 0.6-point F1 loss relative to Gemma4 31B.
- **Efficiency frontier.** Granite4 3B (few-shot) achieves Op-F1 0.937—95.5% of Gemma4 31B’s quality—at 46 J and 0.39 s, that is, roughly 40× less energy and 30× lower latency. Llama3.1 8B (0.950, 79 J) and Mistral 7B (0.942, 70 J) offer marginally higher accuracy at modestly higher cost.

Quality saturates rapidly with scale once few-shot prompting is applied: moving from a 3B to a 31B model improves Op-F1 by only ~ 0.044 while increasing energy by a factor of roughly 40. For the resource-constrained, edge-oriented retail setting that motivates this work, a 3–8B model with few-shot prompting is therefore the rational default, reserving 20–31B models for accuracy-critical or server-side deployments. Crucially, this recommendation is contingent on the prompt: the same small models are not viable under minimal or extended prompting (Op-F1 0.50–0.80), so the deployment win arises from the prompt–model pairing rather than from the model alone. The same Pareto structure holds

when mean latency replaces energy on the cost axis (Table 4), confirming that the ranking of configurations is robust to the choice of cost metric.

4.4. Error Analysis

Field-level accuracies localise the residual errors. Product-span extraction is the hardest field for every model that makes appreciable errors: under the minimal prompt the 3–8B models score only 0.70–0.87 on the product field, below both their action (0.76–0.93) and quantity (0.75–0.93) accuracies, reflecting sensitivity to surface form such as pluralisation; quantity resolution is additionally stressed by idiomatic numerals (“a pair”, “half a dozen”, “a dozen”) that must be mapped to integers. For the near-ceiling large models the three fields all exceed 0.95 and their differences are negligible. The gap between operation-level F1 and whole-command exact match is informative: even Gemma4 31B (few-shot) scores 0.981 Op-F1 but 0.933 exact match, because a command counts as correct only if all of its 3.64 operations on average are simultaneously correct, so per-operation errors compound multiplicatively at the command level. Exact match is consequently the more stringent and deployment-relevant metric for transactional settings, where a single misparsed item corrupts the entire cart update.

5. Limitations and Future Work

6. Conclusions

This section is not mandatory, but can be added to the manuscript if the discussion is unusually long or complex.

Author Contributions: For research articles with several authors, a short paragraph specifying their individual contributions must be provided. The following statements should be used “Conceptualization, X.X. and Y.Y.; methodology, X.X.; software, X.X.; validation, X.X., Y.Y. and Z.Z.; formal analysis, X.X.; investigation, X.X.; resources, X.X.; data curation, X.X.; writing—original draft preparation, X.X.; writing—review and editing, X.X.; visualization, X.X.; supervision, X.X.; project administration, X.X.; funding acquisition, Y.Y. All authors have read and agreed to the published version of the manuscript.”, please turn to the [CRediT taxonomy](#) for the term explanation. Authorship must be limited to those who have contributed substantially to the work reported.

Funding: This work was (partially) supported by the EC Digital Europe Programme EDIH Adria 2.0 (101256325); SPIN projects IP.1.1.03.0120, IP.1.1.03.0158 and IP.1.1.03.0039; and NextGenerationEU University grants: uniri-iz-25-6, uniri-iz-25-220, IIP_010144, IIP_010136

Data Availability Statement: The synthetic evaluation corpus, the generation-and-evaluation harness (`generate_fast.py`), and the plotting scripts used to produce the figures and tables in this paper are available from the authors and will be deposited in a public repository upon publication.

Acknowledgments: This research was performed using the Advanced computing service provided by University of Zagreb University Computing Centre - SRCE.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

LLMs	Large Language Models
NLP	Natural Language Processing
PEFT	Parameter-Efficient Fine-Tuning
LoRA	Low-Rank Adaptation
IA ³	Infused Adapter by Inhibiting and Amplifying Inner Activations
ICL	In-Context Learning
JSON	JavaScript Object Notation
SFT	Supervised Fine-Tuning
F1	F1 Score
GPU	Graphics Processing Unit
DDP	Distributed Data Parallel
MLP	Multi-Layer Perceptron
bf16	BFloat16 Precision Format
tf32	TensorFloat-32

Appendix A. Prompt Templates

The three system prompts are reproduced verbatim below. In every configuration the user turn contains only the raw command; the system prompt is the sole experimental manipulation.

Appendix A.1. Minimal Prompt

```
You are an assistant that analyzes user input related to a shopping cart. Your task is to extract cart
operations from the user's message.
Return ONLY a JSON array where each item has:
- action (one of: 'add', 'remove')
- product (the product name exactly as the user mentions it)
- quantity (an integer; if not specified, assume 1)
Output format:
[{"action": "add", "product": "...", "quantity": 1}]
```

Appendix A.2. Extended Prompt

```
You are a shopping cart assistant. Your task is to extract cart operations from the user's message:
- 'action': classify the intent as either 'add' or 'remove', even if the user uses other words or
  phrases (e.g., 'put', 'insert', 'delete', 'take out', 'nix', 'ring up', 'pack in', 'skip' etc.).
- product: the exact product name as mentioned by the user
- quantity: an integer (default to 1 if not specified).
Return only a valid JSON array of objects, with no explanations, formatting, or extra text. The output
must follow this format exactly:
[{"action": "...", "product": "...", "quantity": ...}]
```

Appendix A.3. Few-Shot Prompt

```
You are a shopping-cart assistant whose only job is to parse the user's request and output a JSON array
where every element has this exact schema:

{
  "action": "<add|remove>",
  "product": "<exact product name>",
  "quantity": <integer>
}

Rules:
1. "action" must be either "add" or "remove". Map any synonyms ("put in", "insert", "take out", "nix",
   "delete", etc.) to these two.
2. "product" is exactly what the customer wants, stripped of any action words or numbers.
3. "quantity" is an integer. If the user does not specify a number, default to 1.
4. Output ONLY the JSON array - no markdown, no explanations, no extra keys or text.

Examples:
```

```
User: Toss in 14 razors in there, throw in 20 pies, load up with 19 coffees, and also nix a pair of
      bananas
Output:
[
{"action": "add", "product": "razors", "quantity": 14},
{"action": "add", "product": "pies", "quantity": 20},
{"action": "add", "product": "coffees", "quantity": 19},
{"action": "remove", "product": "bananas", "quantity": 2}
]

User: Yo lose a dozen soaps, stick in a few potatoes in there, then take away a couple wraps
Output:
[
{"action": "remove", "product": "soaps", "quantity": 12},
{"action": "add", "product": "potatoes", "quantity": 3},
{"action": "remove", "product": "wraps", "quantity": 2}
]

User: Add apples
Output:
[
{"action": "add", "product": "apples", "quantity": 1}
]

Now parse the user's next message.
```

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.